

Sliding Window Algorithms

A senior-interview study guide for fixed windows, variable windows, frequency maps, at-most/exactly transforms, monotonic dequeues, invariants, and the code shapes worth memorizing.

Core Mental Model

Sliding window is a technique for scanning contiguous regions of an array or string without reprocessing the same elements. You move the right pointer to include new information, and sometimes move the left pointer to restore a rule. The window is always the current interval from left to right.

Memorize

The reusable shape is: expand right, update state, shrink left while invalid, update answer. For fixed windows, shrink exactly one old element after the window reaches size k .

Question	Meaning	Algorithm impact
Is the answer contiguous?	Subarray or substring, not subsequence	Sliding window may apply
Is window size fixed?	Exactly k elements/chars	Use fixed window
Is there a validity rule?	At most k , $\text{sum} \leq \text{target}$, no duplicates	Use variable window
Are values nonnegative?	Adding right only increases sum	Two-pointer shrinking is valid
Need max/min inside window?	Extreme changes as endpoints move	Use monotonic deque

What Makes It Senior-Level

The senior-level skill is not remembering 30 variants. It is naming the invariant and knowing when the pattern is invalid. Sliding window works when advancing the right pointer changes the state monotonically enough that moving left can repair the violation. If negatives, arbitrary ordering, or non-contiguous choices break that property, you may need prefix sums, binary search, DP, or a heap instead.

1. Fixed-Size Windows

Fixed-size window problems ask for a property of every contiguous block of length k . The key is to reuse the previous window by adding the incoming element and removing the outgoing element.

Memorize Fixed window invariant: after processing index right , the maintained state describes exactly the last k items, once $\text{right} \geq k - 1$.

```
def fixed_window_sum(nums, k):
    window = sum(nums[:k])
    best = window

    for right in range(k, len(nums)):
        window += nums[right] - nums[right - k]
        best = max(best, window)

    return best
```

Problem cue	State	Answer update
Maximum average subarray	sum of k	max sum / k
Contains duplicate within k	set or counts	check before/after maintaining size
Permutation in string	char counts over length $\text{len}(\text{pattern})$	compare counts
Find all anagrams	char counts over pattern length	record left index on match
Repeated DNA sequences	substring/hash of length 10	seen counts

Two Equivalent Code Shapes

You can prefill the first k elements, or use one loop and remove the outgoing item when the window grows past k . The one-loop shape generalizes better when state maintenance is more complex.

```
# One-loop fixed window
state = empty_state()
for right, x in enumerate(items):
    add(x)

    if right >= k:
        remove(items[right - k])

    if right >= k - 1:
        update_answer()
```

Common Mistake

Do not update the answer before the window has size k unless the problem explicitly allows smaller windows. Fixed means fixed.

2. Variable Windows

Variable windows grow and shrink to maintain a validity condition. The right pointer explores new candidates. The left pointer removes old elements until the window becomes valid again.

Memorize Variable window invariant: after the shrink loop finishes, the current window is valid. Then update the answer using $\text{right} - \text{left} + 1$ or another state-derived value.

```
def longest_valid(items):
    left = 0
    best = 0
    state = empty_state()

    for right, x in enumerate(items):
        add(x, state)

        while invalid(state):
            remove(items[left], state)
            left += 1

        best = max(best, right - left + 1)

    return best
```

Pattern	Validity rule	Example
Longest no repeats	all counts ≤ 1	Longest Substring Without Repeating Characters
At most k distinct	distinct_count $\leq k$	Longest Substring with At Most K Distinct
Max consecutive ones III	zeros $\leq k$	Flip at most k zeros
Fruit into baskets	distinct fruit types ≤ 2	At most two categories
Frequency replacement	window_len - max_freq $\leq k$	Replacement budget

The Shrink Loop

The shrink loop should repair exactly the condition you broke by adding right. If the condition is counts, decrement counts. If the condition is sum, subtract `nums[left]`. If the condition is duplicate-free, remove until the duplicate is gone.

When to Use while vs if

Use while when adding one item can leave the window invalid by more than one left move. Use if only when you intentionally keep a fixed-size or near-fixed-size window, or when the invariant guarantees one removal is enough.

3. Frequency Maps and Sets

Most string sliding windows are really frequency-map problems. A count map lets you know whether the window has duplicates, too many distinct characters, enough required characters, or the same multiset as another string.

Memorize Count-map maintenance: increment when right enters, decrement when left leaves, delete keys at zero. Deleting zero-count keys keeps `len(counts)` meaningful.

```
from collections import defaultdict

counts = defaultdict(int)
left = 0
best = 0

for right, ch in enumerate(s):
    counts[ch] += 1

    while len(counts) > k:
        old = s[left]
        counts[old] -= 1
        if counts[old] == 0:
            del counts[old]
        left += 1

    best = max(best, right - left + 1)
```

State	Useful for	Invariant
set	no duplicates in window	items in set are exactly window contents
dict counts	duplicates/distinct/anagrams	counts match current window
need/have counters	minimum covering window	have counts satisfied required chars
max_freq	replacement budget	most common char count in current or historical window

Minimum Window Substring

Minimum-window problems invert the usual longest-valid shape. You expand until valid, then shrink while still valid to minimize. The invariant is: when valid, every required character count is satisfied.

```
need = Counter(t)
window = defaultdict(int)
have = 0
required = len(need)
left = 0
best = (float("inf"), None, None)

for right, ch in enumerate(s):
    window[ch] += 1
    if ch in need and window[ch] == need[ch]:
        have += 1

    while have == required:
        if right - left + 1 < best[0]:
            best = (right - left + 1, left, right)

        old = s[left]
        window[old] -= 1
        if old in need and window[old] < need[old]:
            have -= 1
        left += 1
```

4. At Most, At Least, and Exactly

Many counting problems become easier when expressed as 'at most'. The classic transform is: exactly k equals `at_most(k)` minus `at_most(k - 1)`. This works when every subarray counted by `at_most(k - 1)` is also counted by `at_most(k)`.

Memorize Exactly $K = \text{atMost}(K) - \text{atMost}(K - 1)$. This is one of the highest-value sliding window tricks.

```
def at_most_k_distinct(nums, k):
    if k < 0:
        return 0
    left = 0
    counts = defaultdict(int)
    ans = 0

    for right, x in enumerate(nums):
        counts[x] += 1

        while len(counts) > k:
            old = nums[left]
            counts[old] -= 1
            if counts[old] == 0:
                del counts[old]
            left += 1

        ans += right - left + 1
    return ans

exactly_k = at_most_k_distinct(nums, k) - at_most_k_distinct(nums, k - 1)
```

Why `ans += right - left + 1` Works

After shrinking, every subarray ending at `right` and starting anywhere from `left` through `right` is valid. There are `right - left + 1` of them. This counts subarrays, not maximum length.

Problem	Transform
Subarrays with K distinct	<code>at_most(k) - at_most(k - 1)</code>
Binary subarrays with sum goal	<code>at_most(goal) - at_most(goal - 1)</code> , because values are nonnegative
Nice subarrays with K odds	Convert odd numbers to 1, evens to 0, then exact sum
Count substrings containing constraints	Sometimes <code>total - invalid</code> or <code>at_least</code> via complement

At Least

For 'at least' problems, think complement or shrink-while-valid. Sometimes count all subarrays and subtract those with at most `k - 1`. Other times, once a window is valid, every extension to the right remains valid, so you can add `n - right`.

5. Sums, Products, and Monotonicity

Sliding window works beautifully for sums and products when the input is nonnegative or positive. The reason is monotonicity: adding right cannot decrease the sum/product, and removing left cannot increase it.

Memorize If numbers can be negative, normal sum-based sliding window usually breaks. Reach for prefix sums, hash maps, or dynamic programming instead.

```
# Count subarrays with product less than k, nums are positive
if k <= 1:
    return 0

prod = 1
left = 0
ans = 0

for right, x in enumerate(nums):
    prod *= x
    while prod >= k:
        prod //= nums[left]
        left += 1
    ans += right - left + 1
```

Cue	Works with sliding window?	Reason
All positive product < k	Yes	Product grows when expanded
Nonnegative sum <= k	Yes	Sum grows when expanded
Contains negative numbers	Usually no	Adding right can shrink sum and removing left can grow it
Exact sum with arbitrary ints	No	Use prefix sum + hash count
Maximum subarray sum arbitrary ints	No	Use Kadane or prefix reasoning

Connection to Prefix Sums

Sliding window is often a faster special case of prefix reasoning. Prefix sums handle arbitrary jumps and negative values. Sliding window wins when monotonicity lets left move only forward and never reconsider old starts.

6. Monotonic Deque Windows

A monotonic deque maintains candidates for the maximum or minimum inside the current window. It is the standard solution for sliding window maximum and related range constraint problems.

Memorize	For max deque: remove indices from the back while their values are $\leq$ incoming value; remove front if it falls out of the window; front is the max.
-----------------	---

```
from collections import deque

def max_sliding_window(nums, k):
    dq = deque() # stores indices, values decreasing
    ans = []

    for right, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(right)

        if dq[0] <= right - k:
            dq.popleft()

        if right >= k - 1:
            ans.append(nums[dq[0]])

    return ans
```

Why It Works

If a newer value is greater than or equal to an older value, the older value can never become the maximum while the newer value remains in the window. The older value is dominated and can be discarded.

Problem cue	Deque role
Sliding window maximum	Track max candidate indices
Longest subarray with $\text{abs}(\text{max}-\text{min}) \leq \text{limit}$	Use one max deque and one min deque
Shortest subarray with sum at least k	Use prefix sums plus increasing deque
Constrained subsequence sum	DP values with max deque over recent indices

Senior-Level Distinction

Monotonic deque is still sliding-window thinking, but the state is no longer a simple count or sum. It is a compressed set of undominated candidates.

7. Pattern Recognition

Phrase in prompt	Likely pattern	Watch out
contiguous subarray/substring	Sliding window candidate	If not contiguous, use different tools
length k	Fixed window	Update only after size k
longest ... at most k	Variable window	Shrink while invalid
minimum window containing	Expand until valid, shrink while valid	Need have/need counters
count subarrays with exactly k	At most transform	Count windows, not max length
positive integers and product/sum < k	Variable window count	Requires positivity/nonnegativity
max/min in every window	Monotonic deque	Store indices, not just values
arbitrary negative numbers	Probably prefix sum/hash	Sliding window may be invalid

Decision Map

Start with the output: maximum length, minimum length, count, or list of window values. Then identify the state: sum, product, counts, distinct count, max/min, or required coverage. Finally, decide whether the window has fixed size or validity-based size.

Output type	Common update
Maximum length	best = max(best, right - left + 1) after window is valid
Minimum length	best = min(best, right - left + 1) while window is valid
Count valid subarrays	ans += right - left + 1 after shrink
All matching starts	append(left) or append(right - k + 1) when state matches
Window max values	append(nums[dq[0]]) once right >= k - 1

Problem Drill Set

Stage	Problems	Target internalization
Fixed	Maximum Average Subarray I, Contains Duplicate II, Find All Anagrams	Incoming/outgoing update
Variable basic	Longest Substring Without Repeating Characters, Fruit Into Baskets	Shrink until valid
Budget	Max Consecutive Ones III, Longest Repeating Character Replacement	Violation as budget
Minimum	Minimum Size Subarray Sum, Minimum Window Substring	Shrink while valid
Counting	Subarrays with K Different Integers, Binary Subarrays With Sum	atMost transform
Deque	Sliding Window Maximum, Longest Continuous Subarray With Absolute Diff <= Limit	Undominated candidates
Boundary cases	Subarray Product Less Than K, Prefix Sum Equals K	Know when sliding window fails

8. Memorization Sheet

Pattern	Memorize this
Fixed window	Add incoming, remove outgoing, update when size is k
Variable longest	Expand right, shrink while invalid, update max length after repair
Variable minimum	Expand right until valid, shrink while valid, update min length before breaking validity
Counts	Increment right char, decrement left char, delete at zero
At most count	After repair, $ans += right - left + 1$
Exactly k	$atMost(k) - atMost(k - 1)$
No repeats	Shrink until count of duplicate is back to one
At most k distinct	Shrink while $len(counts) > k$
Replacement budget	$window_len - max_freq \leq k$
Positive sum/product	Shrink while threshold violated
Monotonic max deque	Pop smaller/equal from back, expire front, front is max

One-Sentence Invariants

- Fixed window: state represents exactly the last k elements after the outgoing element is removed.
- Variable longest: after the while loop, the current window is valid and is the longest valid window ending at right for this left movement.
- Minimum window: when valid, try to remove left until the window would become invalid.
- At most count: after repair, every start index from left to right forms a valid subarray ending at right.
- Monotonic deque: indices in the deque are inside the window and values are ordered so the front is the current extreme.

What Not To Memorize Blindly

- Do not use sliding window just because a problem says subarray; negative numbers may break it.
- Do not update max length before restoring validity.
- Do not forget to delete zero-count keys when $len(counts)$ matters.
- Do not store only values in a monotonic deque when you need to expire old indices.
- Do not confuse counting windows with finding the longest window; the answer update is different.